Distributed Training of DNNs

Sparsh Mittal

Reference: Sun et al., “Optimizing Network Performance for Distributed DNN Training on GPU Clusters: ImageNet/AlexNet Training in 1.5 Minutes”, Arxiv, 2019.

Acronyms

- FWP = forward propagation
- BWP = backward propagation

Summary of Optimizations used

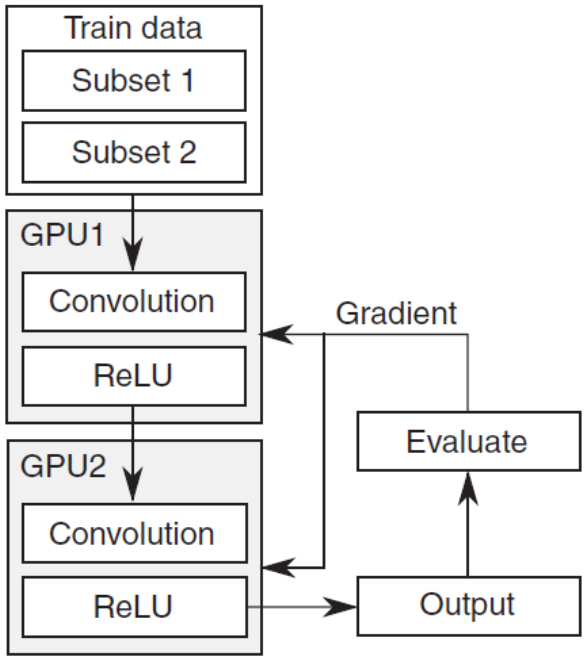
- Use of optimized library
- NVIDIA's Tensor cores
- mixed-precision training
- Pipelining
- Using optimized implementation of allreduce
- Lazy allreduce for improving hardware utilization
- Coarse-Grained Sparse Communication

Background on distributed training

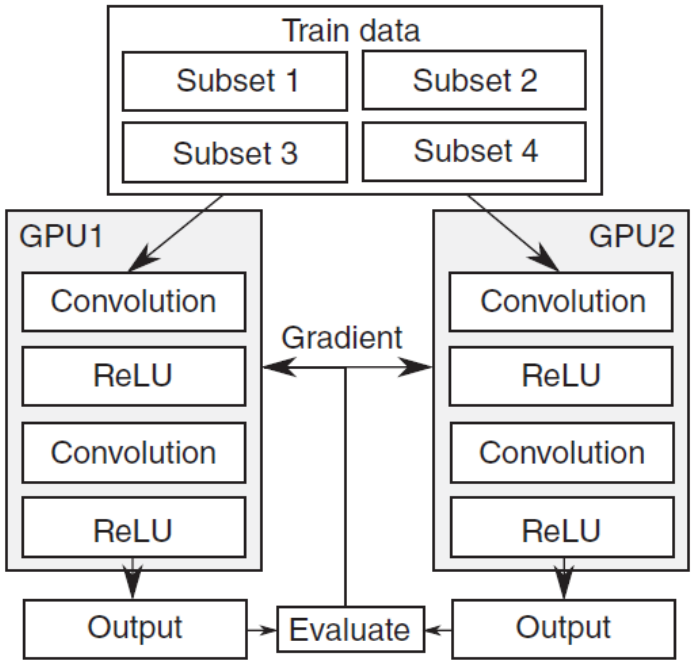
Two Approaches for distributed training

- **Data Parallelism:** same model is replicated on each GPU but is fed with different parts of the training data.
- **Model parallelism:** requires splitting the model across different GPUs and same data is used to train the model.

Model Parallelism

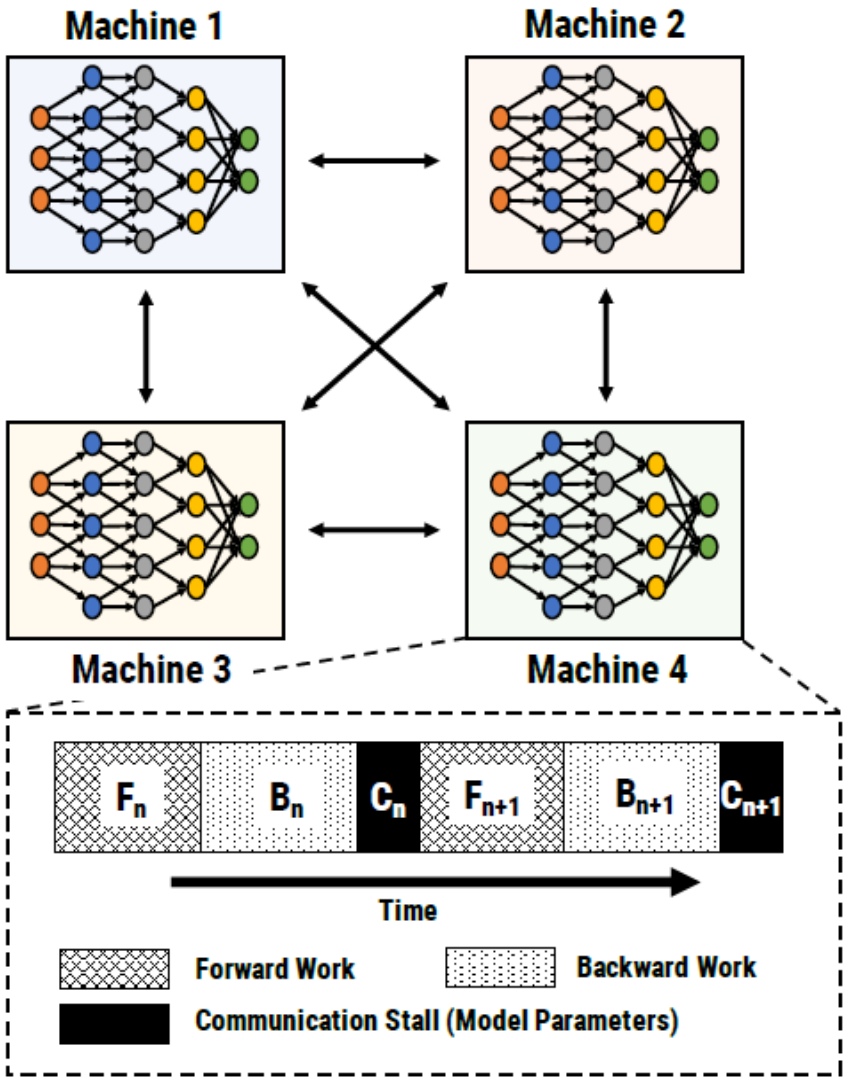


Data Parallelism

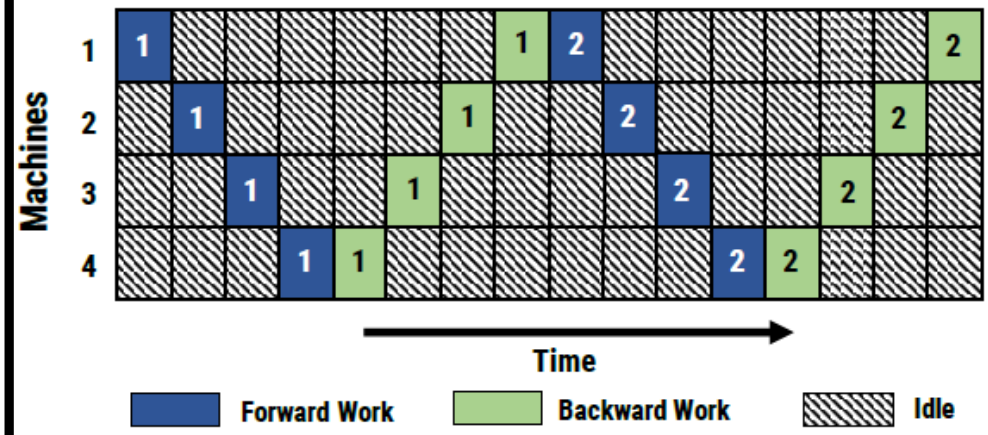


Two Approaches for distributed training

Example data-parallel setup with 4 machines



Model parallel training with 4 machines. Numbers indicate minibatch ID. Assuming communicating activations across machines has no overhead.

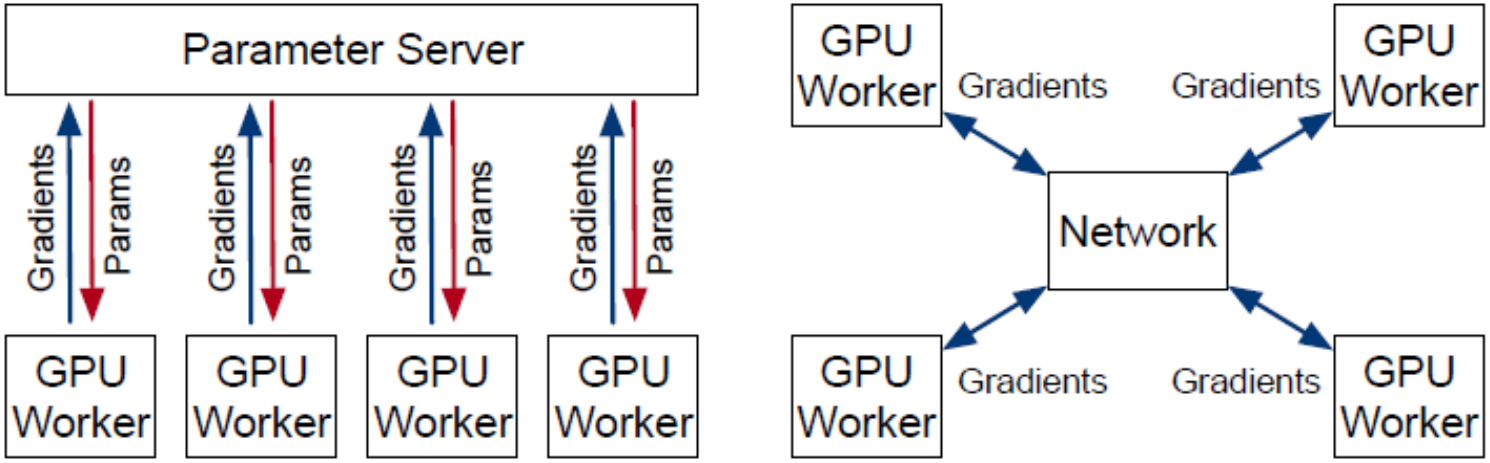


Challenges

- Increasing computation power demands a similar increase in network bandwidth
- But, upgrading datacenter networks is expensive
  - A cluster in 2013 used network bandwidth of 56Gbps
  - A cluster in 2018 used bandwidth of 100Gbps

System architectures for distributed DNN

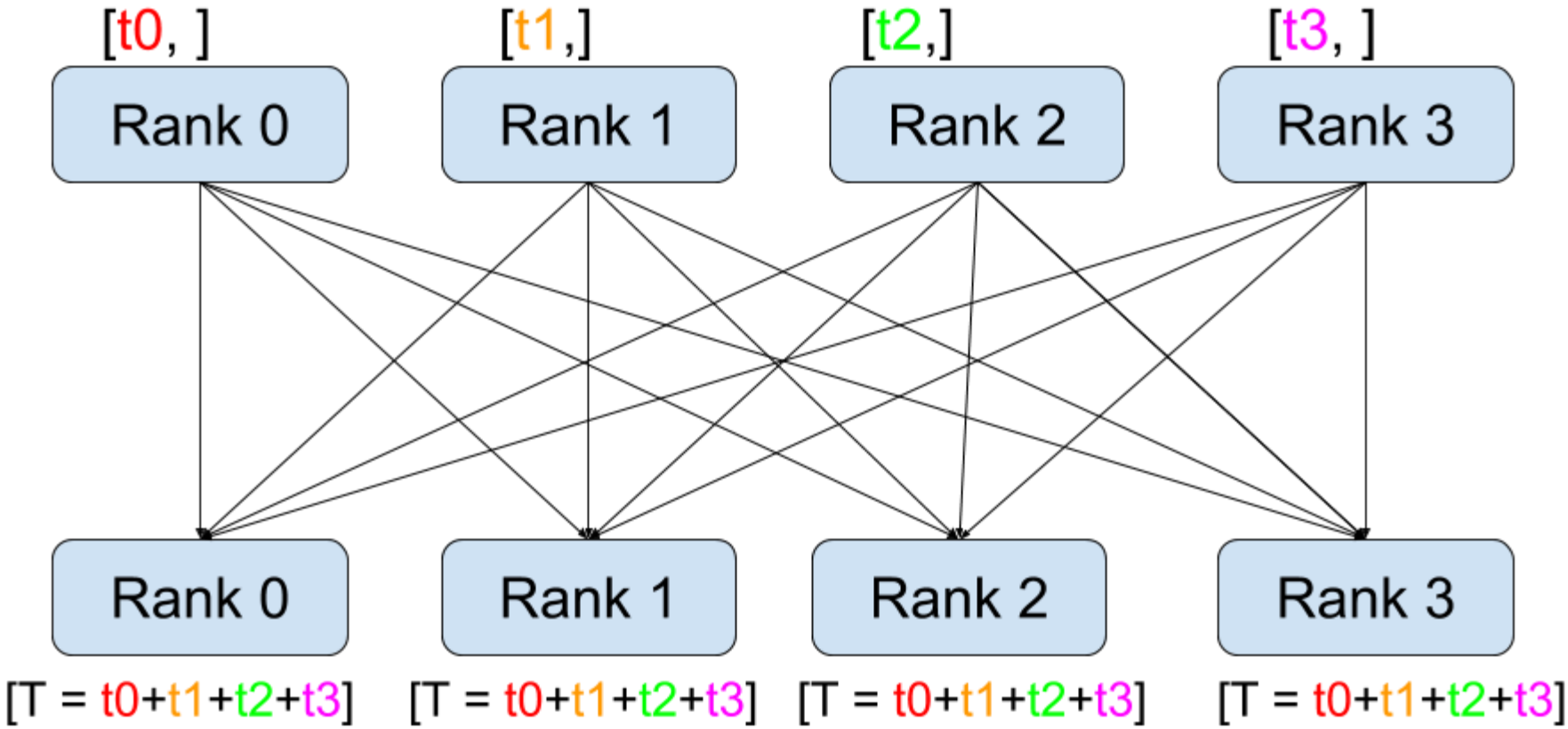
- Master-slave architecture: all GPU workers push local gradients to parameter servers for model update, and pull latest parameters.
- P2P architecture: every GPU worker communicates with each other to exchange gradients using allreduce. Then, every GPU has aggregated gradients, and uses them to update replicated parameters locally.



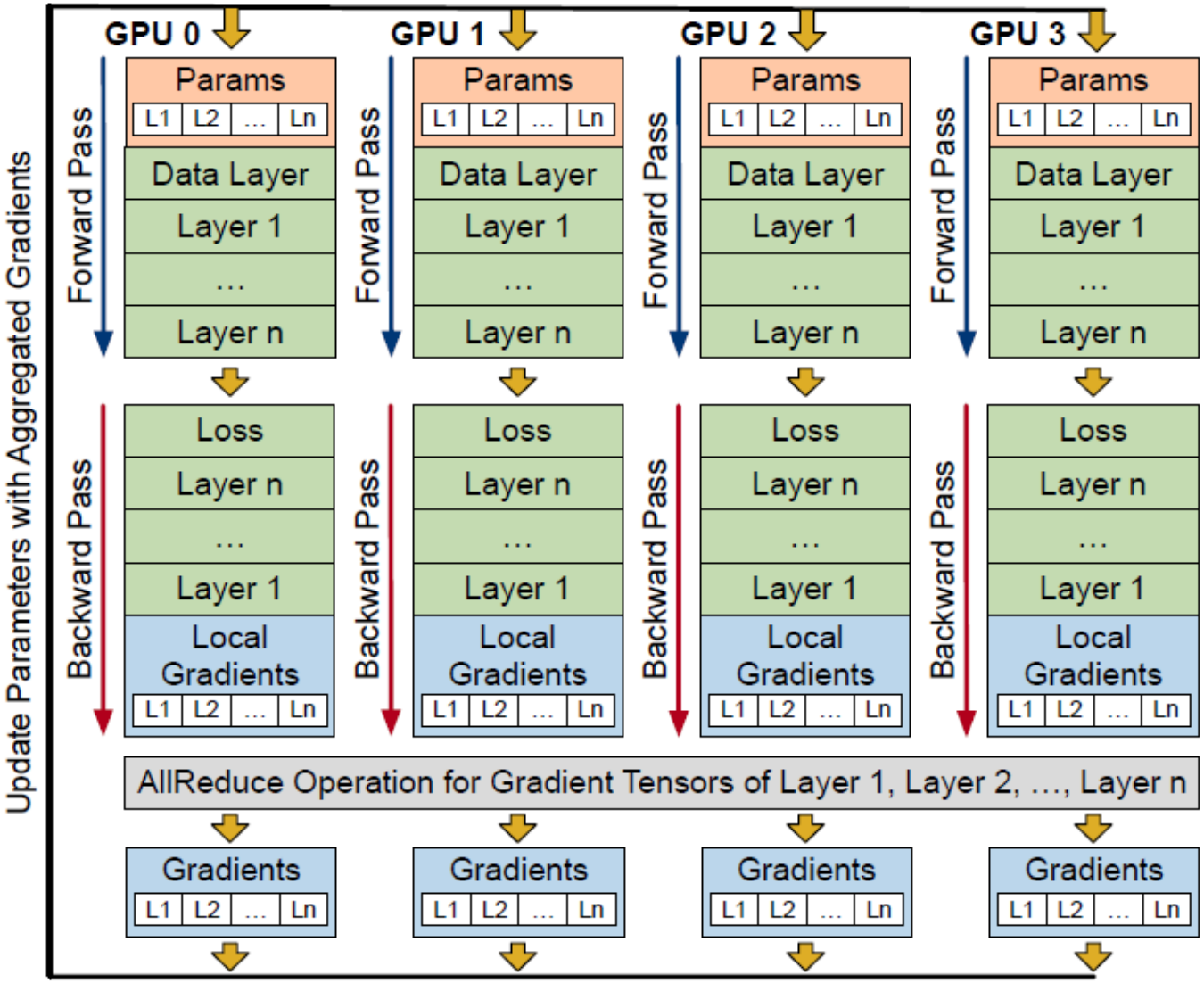
(a) Master-Slave Architecture (Parameter Server Approach)

(b) P2P Architecture (AllReduce Approach)

Allreduce operation



Baseline system for distributed DNN training



Working of distributed training (1 of 2)

- FWP: each GPU processes a batch of training data from layer-1 to layer-n.
- At the end of FWP, each GPU outputs a loss value to measure prediction error.
- Next, GPUs start BWP from layer-n to layer-1.
- On completing BWP computation of layer-i, this layer's gradients are generated for its learnable parameters.

Working of distributed training (2 of 2)

- Not all layers have learnable parameters, such as ReLU layer and pooling Layer.
- Some layers may use multiple tensors to represent its parameters and gradients.
  - CONV layer writes a weight gradient tensor and a bias gradient tensor during BWP.
- After BWP, a set of gradient tensors are generated on every GPU. An allreduce operation is applied to each tensor for gradient aggregation.
- After communication stage, each GPU has summed gradients, which are used to update replicated parameters locally.

Experimentation platform

Hardware Platforms

- Cluster-P contains 16 physical machines and 128 GPUs with Pascal microarchitecture.
- Cluster-V contains 64 physical machines and 512 GPUs with Volta microarchitecture.

Interconnects

- GPUs of a machine are connected by PCIe.
- All machines of a cluster are connected by 56Gbps InfiniBand, and share a distributed file system, which is used for training dataset management.

Frameworks:

- PyTorch-0.4: uses Gloo as communication backend
- System-I: uses OpenMPI as communication backend

- NCCL: library of multi-GPU collective communication primitives, and it contains an efficient implementation of ring-based allreduce.

Workload and their charcteristics

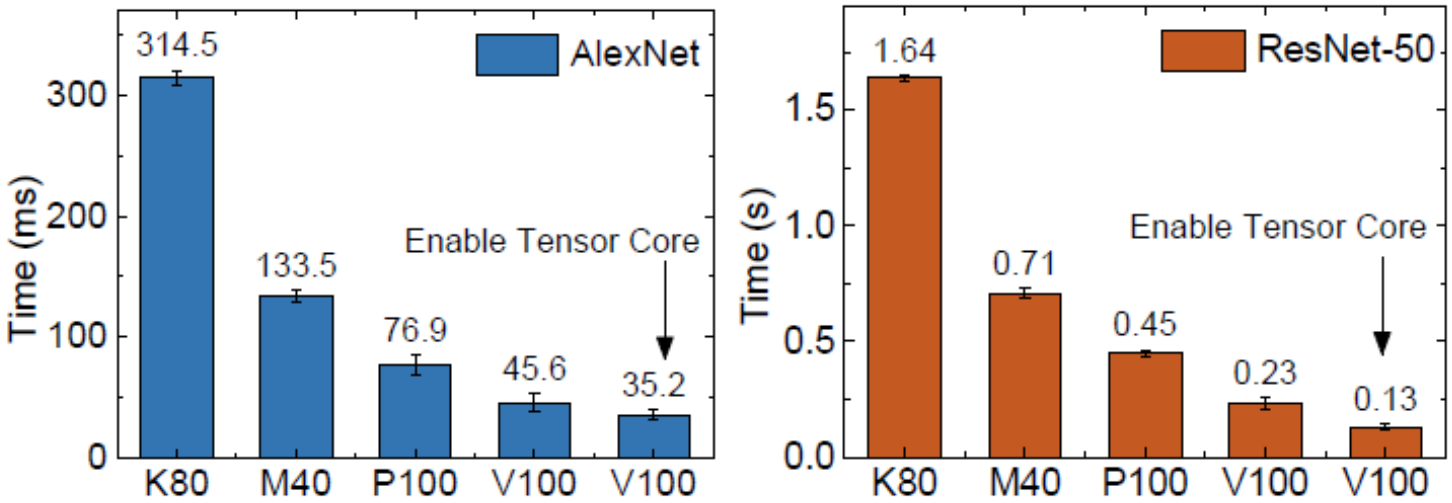
Observation about AlexNet and ResNet

Methods	LeNet-5 [54]	AlexNet [7]	OverFeat (fast) [8]	VGG-16 [9]	GoogLeNet [10]	ResNet-50(v1) [11]
Top-5 errors	n/a	16.4	14.2	7.4	6.7	5.3
Input size	28 × 28	227 × 227	231 × 231	224 × 224	224 × 224	224 × 224
Number of Conv Layers	2	5	5	16	21	50
Filter Size	5	3,5,11	3,7	3	1,3,5,7	1,3,7
Number of Feature Maps	1,6	3–256	3–1024	3–512	3–1024	3–1024
Stride	1	1,4	1,4	1	1,2	1,2
Number of Weights	26 k	2.3 M	16 M	14.7 M	6.0 M	23.5 M
Number of MACs	1.9 M	666 M	2.67 G	15.3 G	1.43 G	3.86 G
Number of FC layers	2	3	3	3	1	1
Number of Weights	406 k	58.6 M	130 M	124 M	1 M	1 M
Number of MACs	405 k	58.6 M	130 M	124 M	1 M	1M
Total Weights	431 k	61 M	146 M	138 M	7 M	25.5 M
Total MACs	2.3 M	724 M	2.8 G	15.5 G	1.43 G	3.9 G

- Parameter size of AlexNet is ~2.5X of ResNet-50
- Computation of ResNet-50 is ~5.6X of AlexNet
- Bottleneck of AlexNet lies in communication, while the bottleneck of ResNet-50 lies in computation.

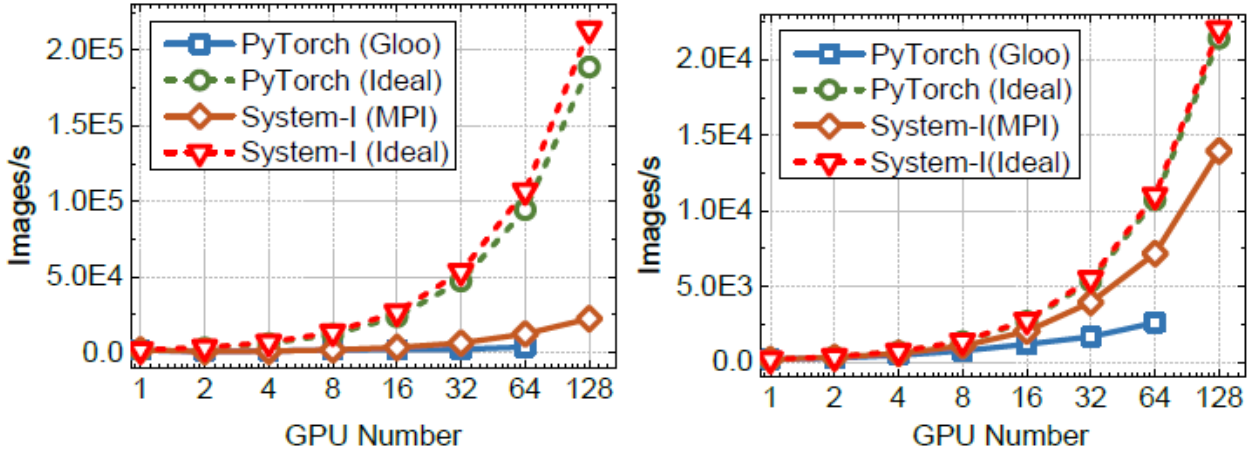
Single-GPU per-iteration training time

For AlexNet (batch size 128) and ResNet-50 (batch size 64) measured with CuDNN-7 on K80, M40, P100 and V100 GPUs, which were launched on 2014, 2015, 2016 and 2017, and use Kepler, Maxwell, Pascal and Volta micro-architectures, respectively.



(a) AlexNet (b) ResNet-50

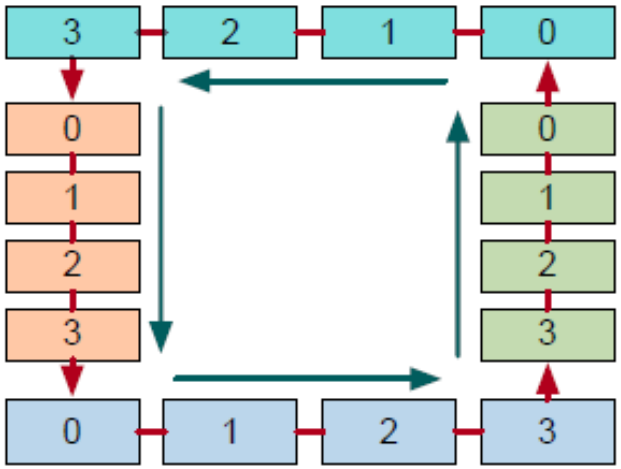
Baseline performance



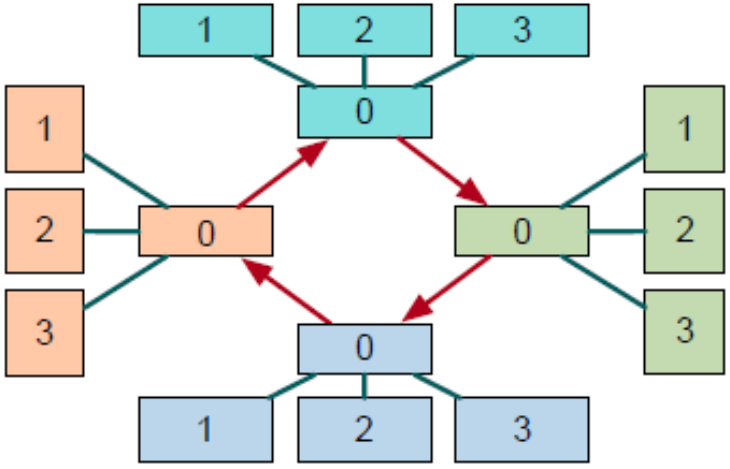
(a) AlexNet (b) ResNet-50

- Due to the high communication cost, baseline system shows poor scalability
- Since ResNet-50 has less learnable parameters and takes longer forward/backward computation time than AlexNet, its communication overhead is not as significant as AlexNet.

Different Implementations of AllReduce



(a) Ring-based AllReduce

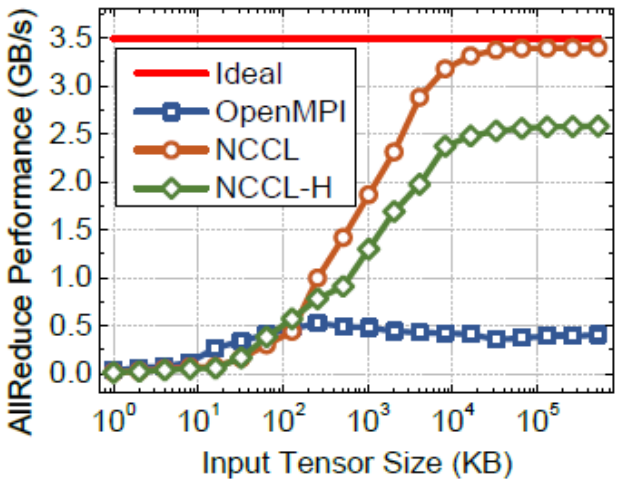


(b) Hierarchical AllReduce

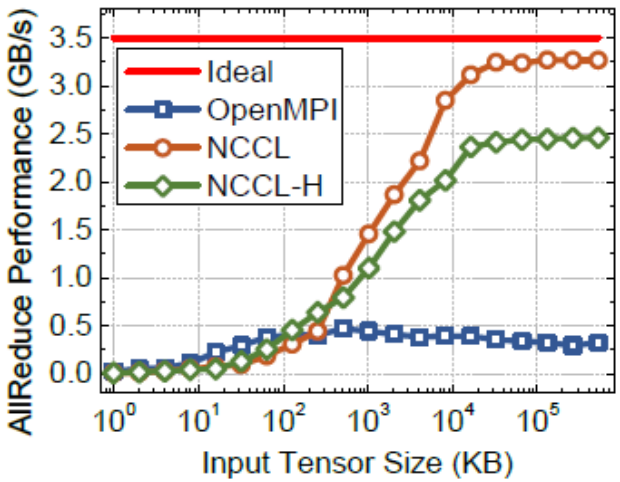
Here, there are 4 machines, each of which has 4 GPUs.
(a): all 16 GPUs are arranged in a logical ring.

b): 16 GPUs are partitioned into 4 groups, and only the master GPU of each group constructs a logical ring for allreduce.

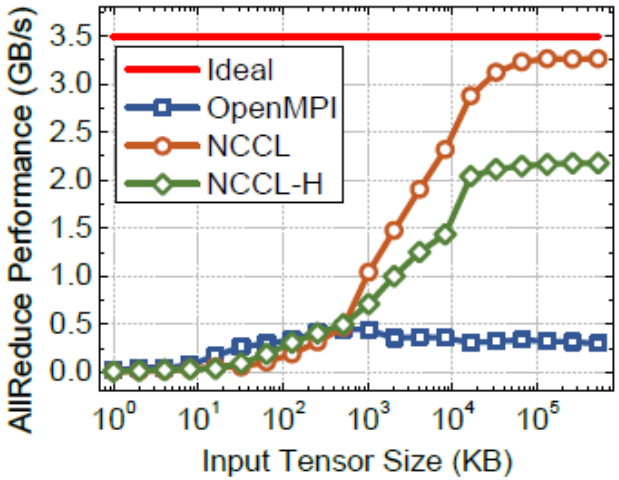
FP32 allreduce on Cluster-P



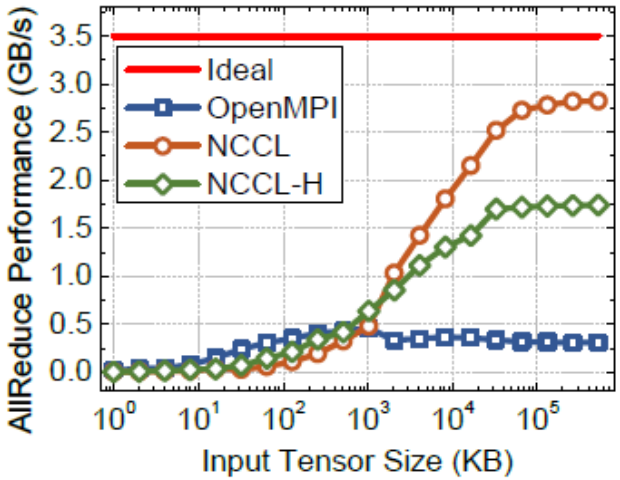
(a) 16-GPU



(b) 32-GPU



(c) 64-GPU

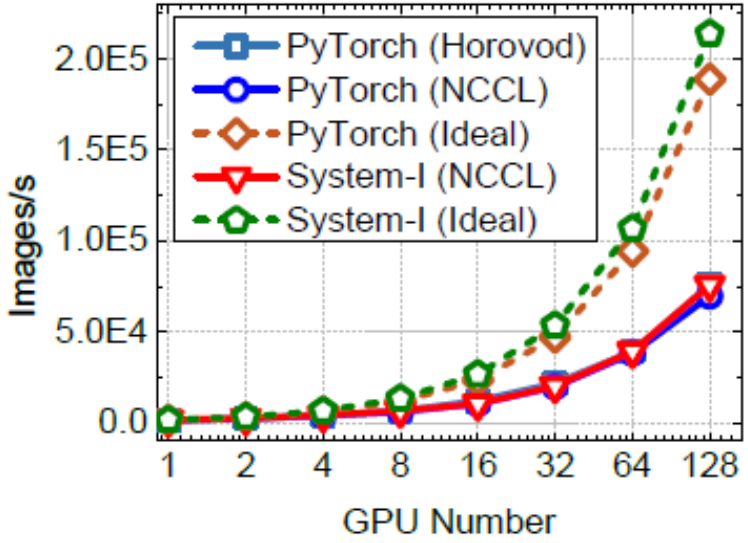


(d) 128-GPU

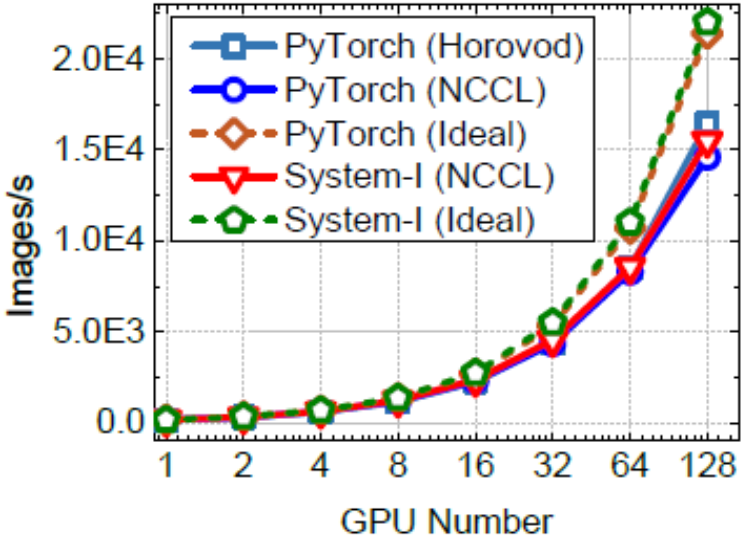
OpenMPI vs NCCL vs NCCL-H

- NCCL-H is implementation of hierarchical allreduce using NCCL APIs
- Compared to OpenMPI, NCCL and NCCL-H improve the performance of allreduce operations.
- When the input tensor size is greater than 1MB, NCCL outperforms NCCL-H in all test cases.
- Intra-group reduce and broadcast operations become the performance bottleneck in NCCL-H, since they are not bandwidth optimal.
- NCCL-H performance can be better in some cases, e.g., when NVLink is used

PyTorch and System-I performance with ring-based allreduce



(b) AlexNet



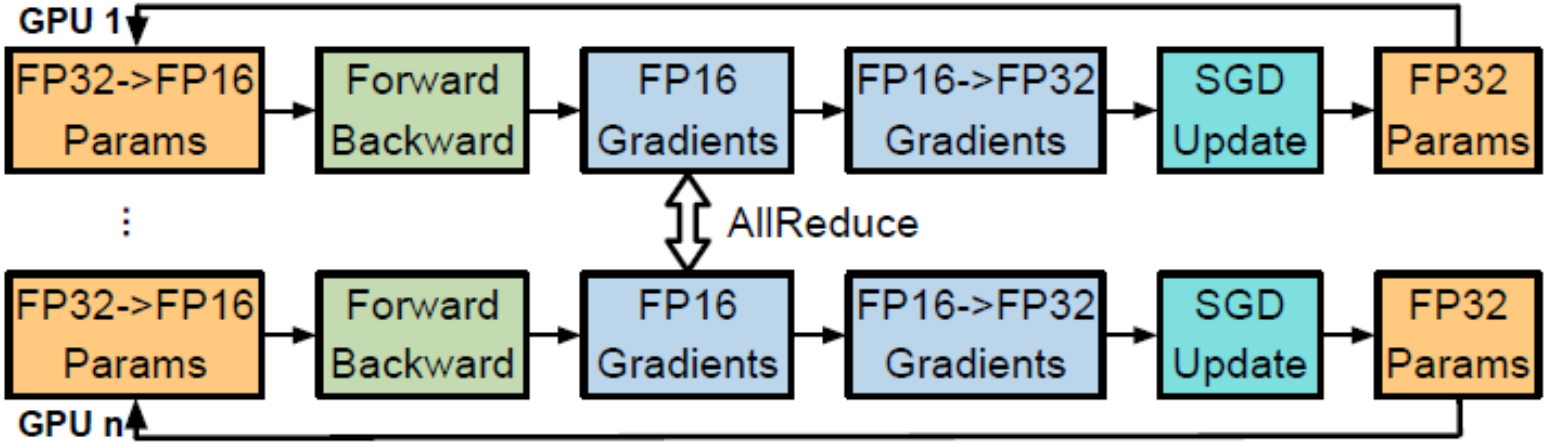
(b) ResNet-50

Even with NCCL, expensive communication operations still significantly reduce the system performance

Horovod is Uber's framework which uses MPI to distribute the computation across multiple worker nodes

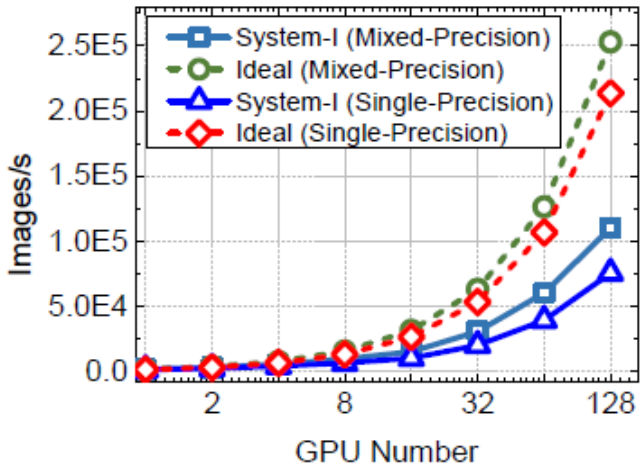
Idea 1: use mixed-precision training

- Uses FP32 values for ``model update'' phase and FP16 parameters and gradients in FWP/BWP phases

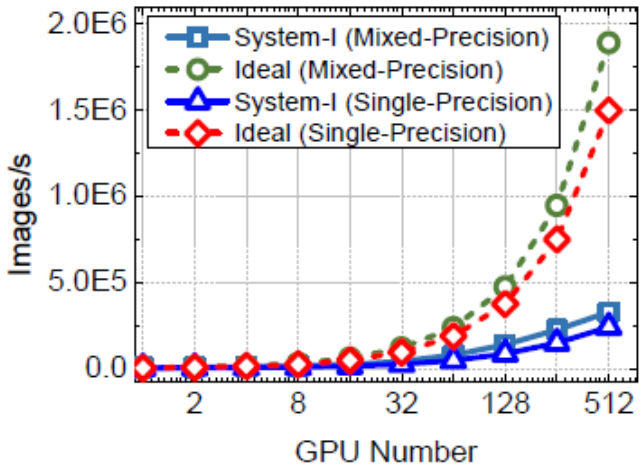


- For FP16 operations, tensor cores can provide between 2x to 8X speedup over FP32 operations

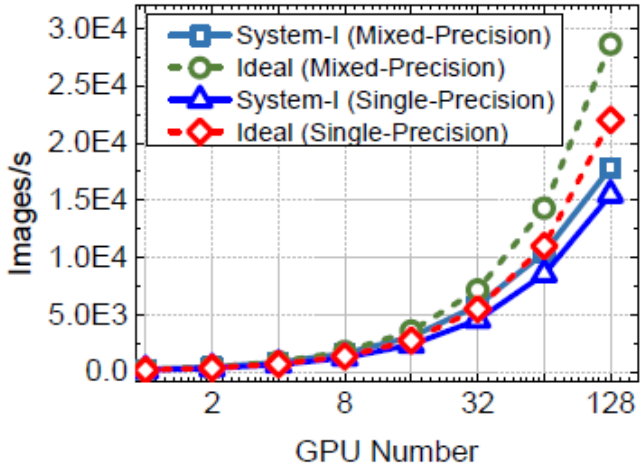
Results with mixed-precision



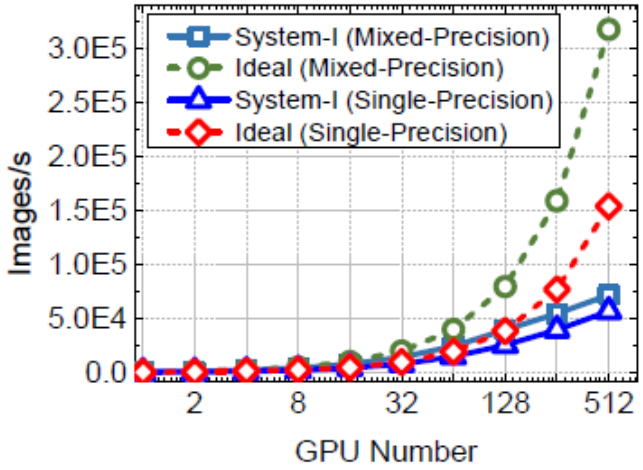
(a) AlexNet on Cluster-P



(b) AlexNet on Cluster-V



(c) ResNet-50 on Cluster-P



(d) ResNet-50 on Cluster-V

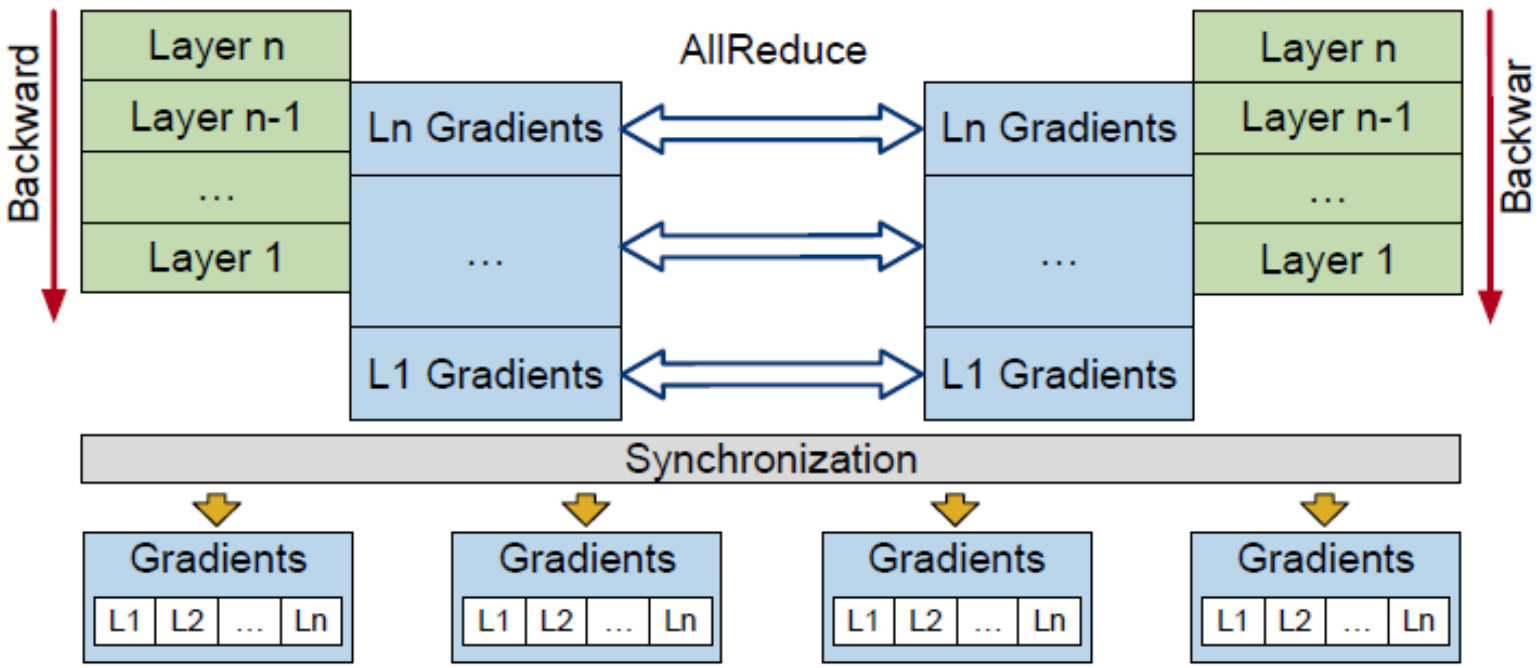
Impact of mixed-precision training

- Compared to FP32 training, mixed-precision training improves system performance on a single GPU.
- Although mixed-precision approach reduces data-transfer overheads and allows doubling the batch-size, the speedup still is much lower than the ideal speedup.
- Mixed-precision training also incurs overhead of conversion between FP16 and FP32, esp for AlexNet.
- Communication overhead is still significant

Idea 2: communication/computation overlap

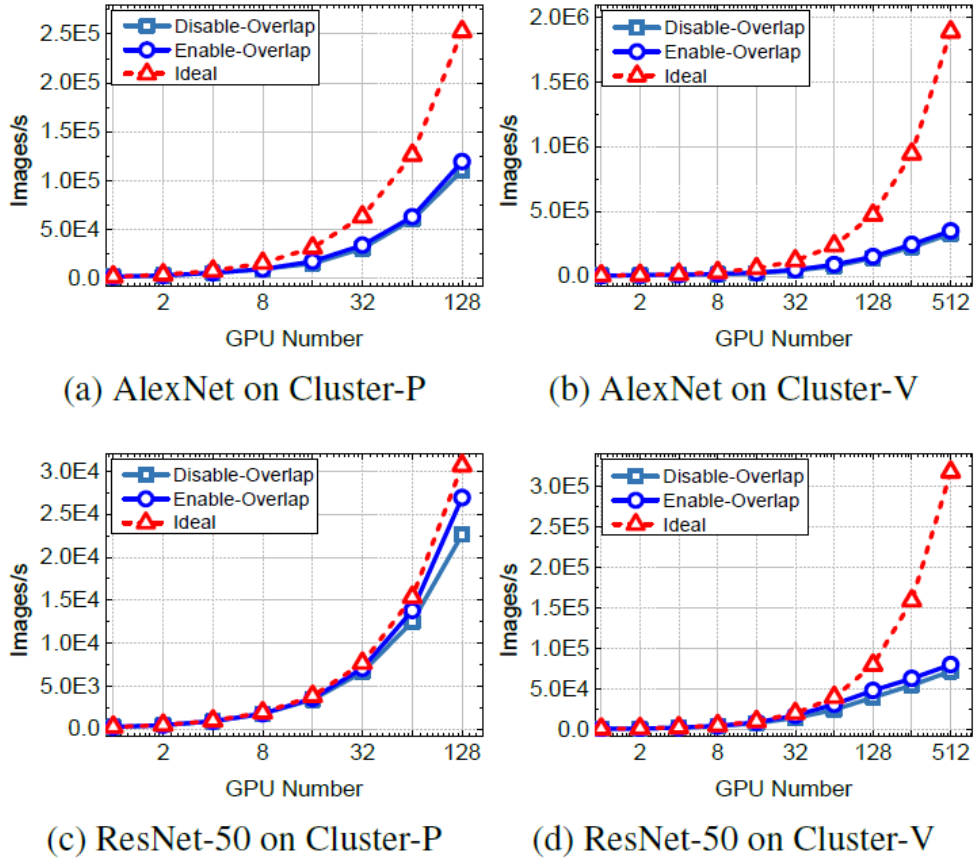
- On completion of BWP for layer-k, its gradient are produced which are not changed during BWP of any layer-j, such that (j<k).
- Hence, they overlap transfer of gradient of layer-k with BWP phase of layer-j (j<k).

Idea 2: communication/computation overlap



Upper layer's communication operations are overlapped with lower layers' computation operations, thus reducing dedicated communication time of an iteration.

Layer-based overlap between computation and communication (mixed-precision, NCCL)



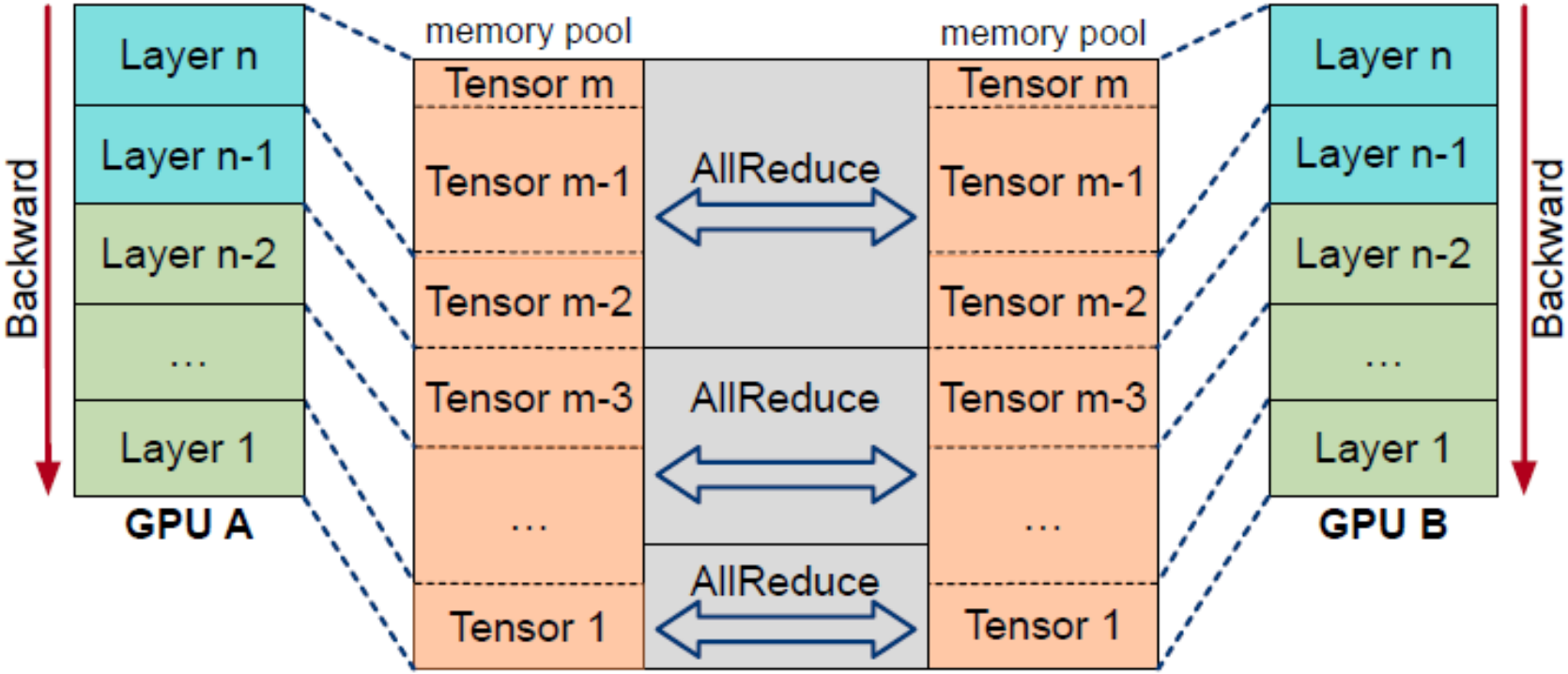
(a) AlexNet on Cluster-P (b) AlexNet on Cluster-V

(c) ResNet-50 on Cluster-P (d) ResNet-50 on Cluster-V

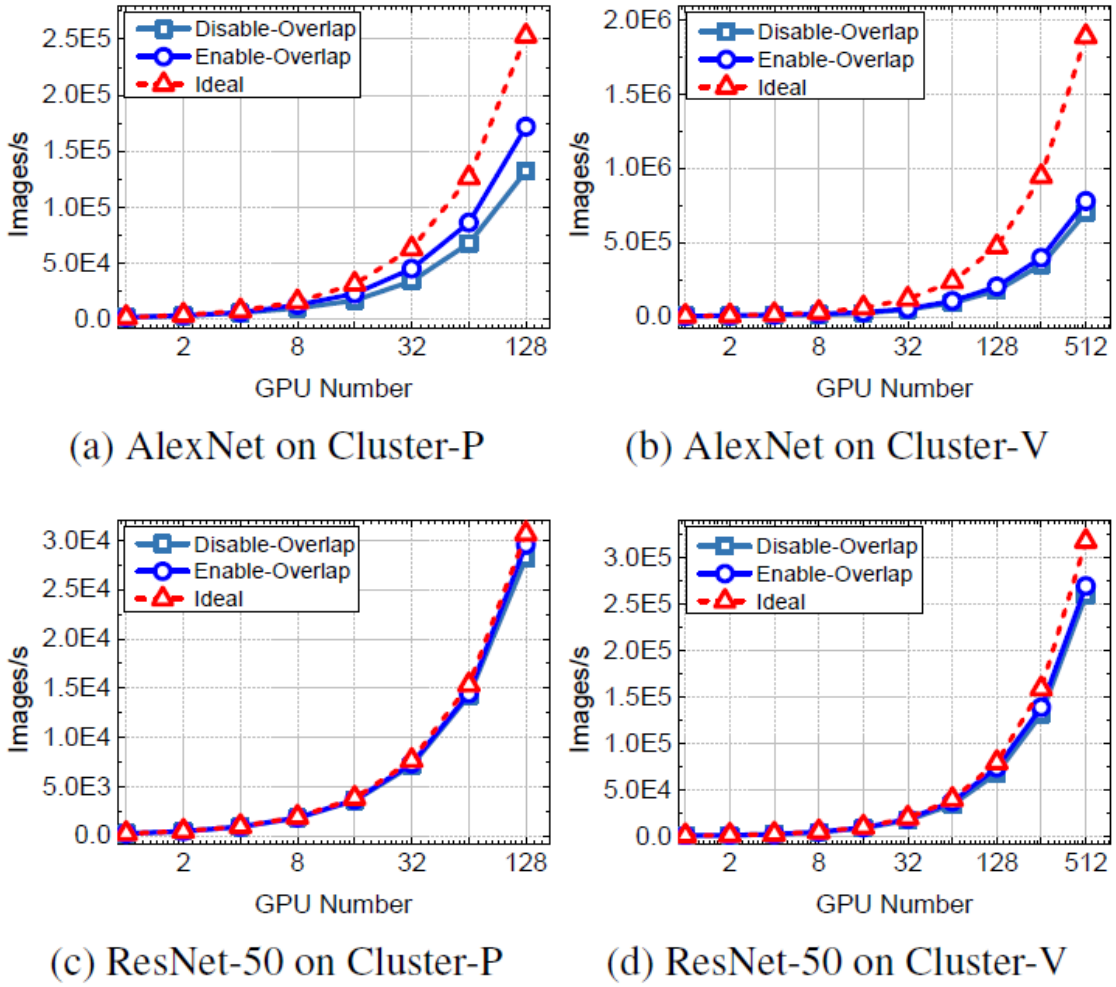
- Still, the speedup remains lower than the ideal speedup. This is because on doing allreduce on small-size gradients, the network is not fully utilized.

Idea 3: Lazy allreduce

- Here, allreduce is not performed immediately when a gradient tensor is generated. Instead, multiple allreduce operations are fused into a single one, thus increasing network throughput with large message size.



System-I performance with NCCL, mixed-precision training and lazy allreduce.



(a) AlexNet on Cluster-P (b) AlexNet on Cluster-V

(c) ResNet-50 on Cluster-P (d) ResNet-50 on Cluster-V

Results with Lazy allreduce

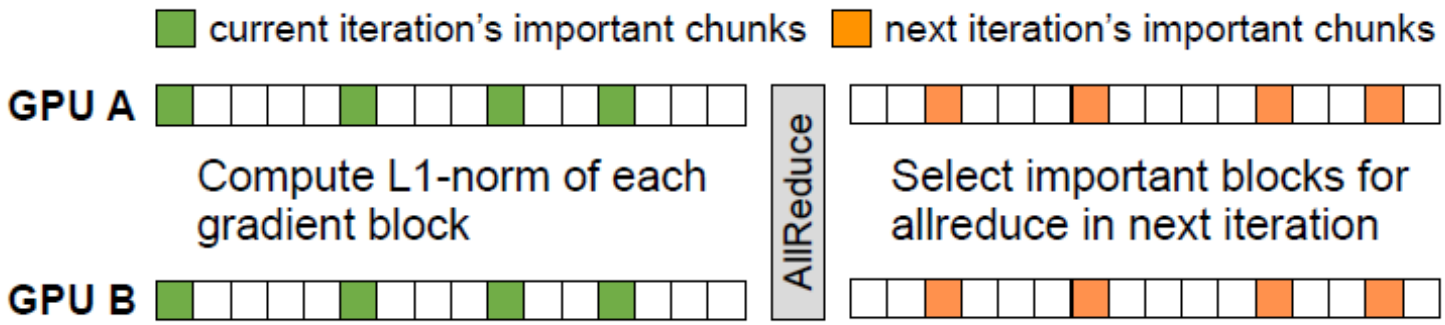
- Laze allreduce significantly improves ResNet-50 training performance
- Since AlexNet generates a huge number of network traffic, System-I still has a huge performance gap with the ideal system

Idea 4: Coarse-Grained Sparse Communication

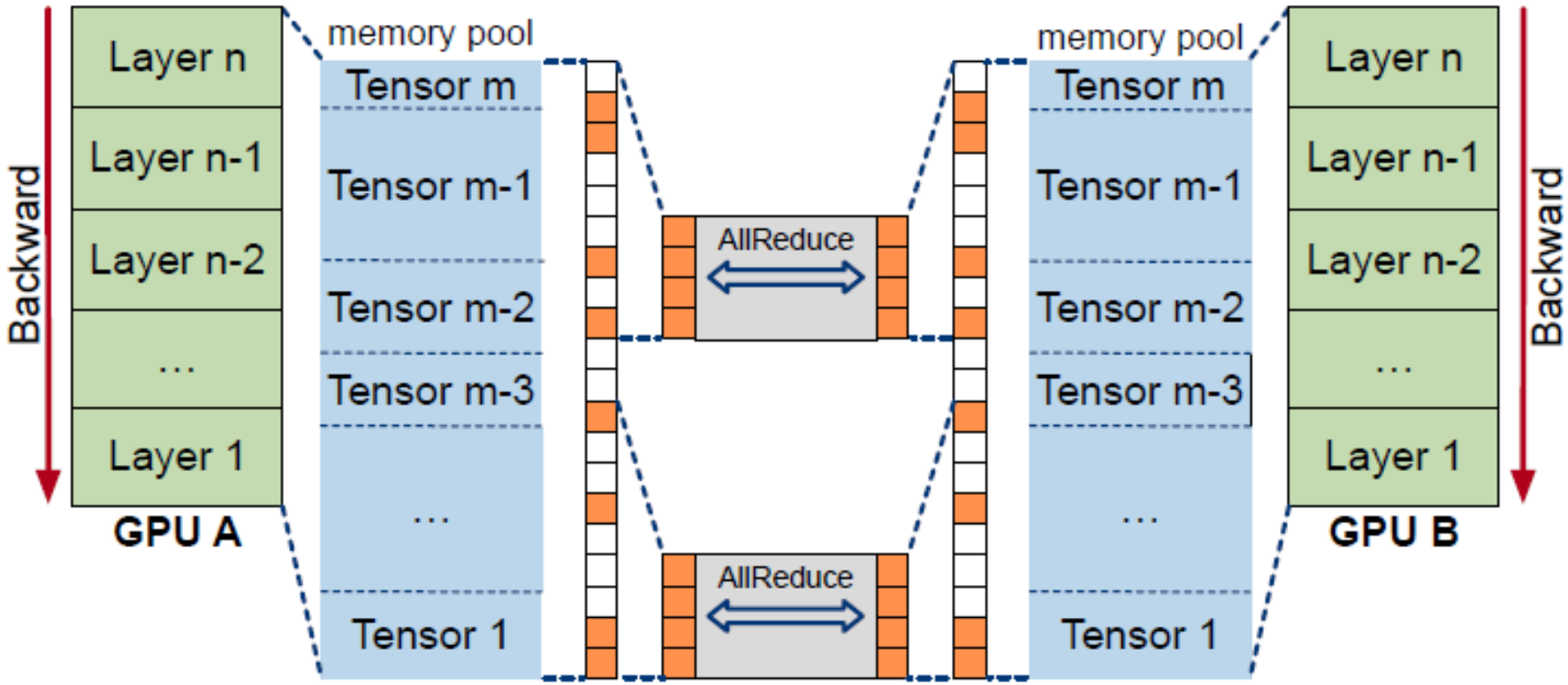
- Deep gradient compression uses fine-grained sparse communication (FCS) to reduce network traffic: only gradients greater than a threshold are transmitted
- FCS reduces up to 99.9% traffic without losing accuracy.
- However, FCS runs sparse allreduce with high computation cost and low network utilization, resulting in no significant gains on their clusters.
- Reason: every GPU transmits different gradients and stores/transmits these values in “index-value” format which is inefficient due to overheads of memory-copy and irregular memory accesses.

Idea 4: Coarse-Grained Sparse Communication

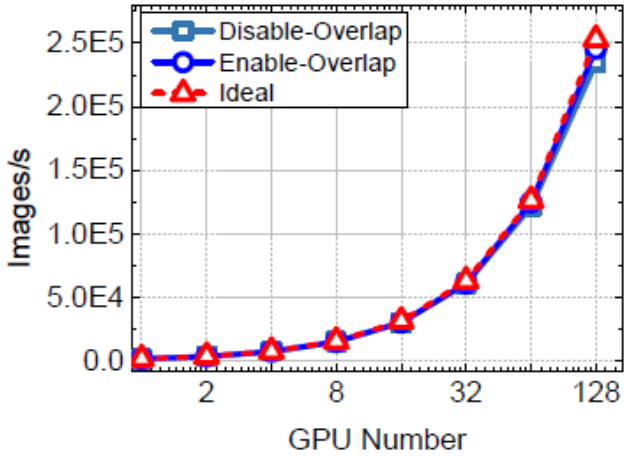
- Gradients in the memory pool are partitioned into chunks having same number (e.g., 32000) of gradients.
- After each iteration, some chunks are chosen as important
- Only important chunks are copied to a buffer and when the buffer size exceeds a threshold, the (lazy) allreduce operation is performed.
- Each GPU selects the same important chunks which allows using NCCL for accumulating and broadcasting those chunks with high bandwidth utilization.



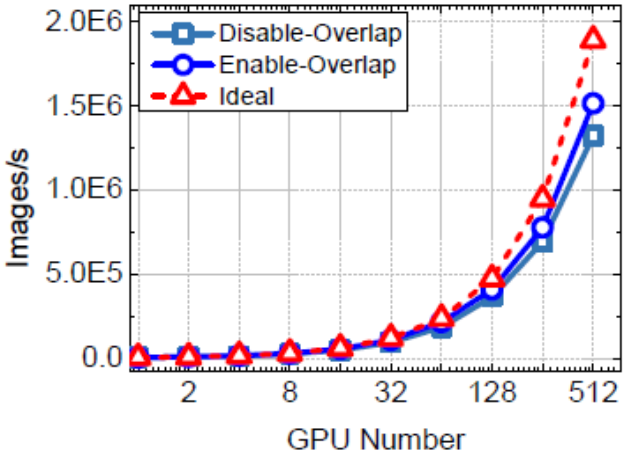
Idea 4: Coarse-Grained Sparse Communication



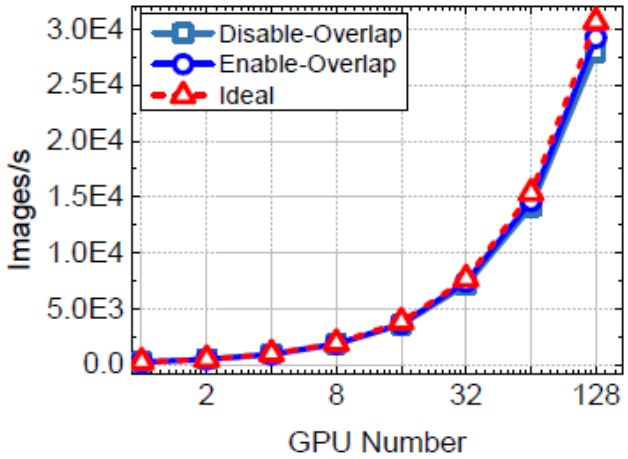
System-I, NCCL, mixed-precision training and coarse-grained sparse communication



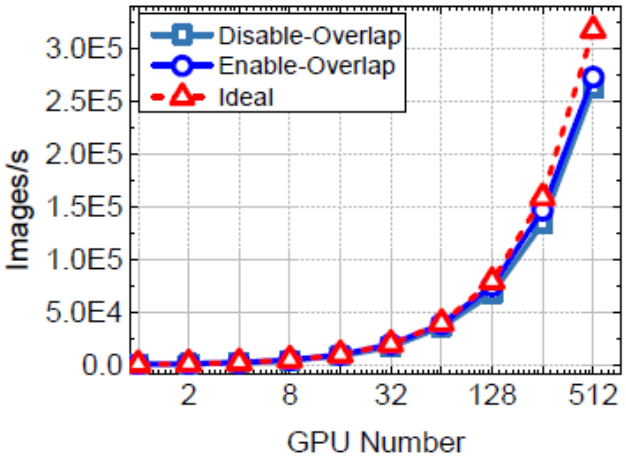
(a) AlexNet on Cluster-P



(b) AlexNet on Cluster-V



(c) ResNet-50 on Cluster-P



(d) ResNet-50 on Cluster-V

- They also use some technique for compensating for loss in model accuracy

Contribution of each technique

AlexNet training throughput (image/s)

	MPI	NCCL	NCCL+MP	NCCL+MP +Overlap	NCCL+MP +LA+Overlap	NCCL+MP+LA +CSC+Overlap
Cluster-P	22.3K (1x)	75.7K (3.4x)	110.3K (4.9x)	119.5K (5.4x)	172.8K (7.7x)	245.4K (11.0x)
Cluster-V	56.2K (1x)	240.0K (4.3x)	326.7K (5.8x)	349.1K (6.2x)	780.3K (13.9x)	1514.3K (26.9x)

ResNet-50 training throughput (image/s)

	MPI	NCCL	NCCL+MP	NCCL+MP +Overlap	NCCL+MP +LA+Overlap	NCCL+MP+LA +CSC+Overlap
Cluster-P	13.9K (1x)	15.5K (1.1x)	17.8K (1.3x)	26.9K (1.9x)	29.6K (2.1x)	29.6K (2.1x)
Cluster-V	30.2K (1x)	56.8K (1.9x)	71.8K (2.4x)	80.0K (2.6x)	269.5K (8.9x)	273.2K (9.0x)

MP: Mixed-Precision Training; LA: Lazy AllReduce; CSC: Coarse-Grained Sparse Communication; Overlap: Computation/Communication Overlap

End-to-End Training Time

AlexNet

	Batch Size	Processor	GPU Interconnect	Time	Top-1 Accuracy
You et al. [40]	512	DGX-1 station	NVLink	6 hours 10 mins	58.8%
You et al. [40]	32K	CPU x 1024	-	11 mins	58.6%
Jia et al. [18]	64K	Pascal GPU x 1024	100 Gbps	5 mins	58.8%
Jia et al. [18]	64K	Pascal GPU x 2048	100 Gbps	4 mins	58.7%
This Work (DenseCommu)	**64K**	**Volta GPU x 512**	**56 Gbps**	**2.6 mins**	**58.7%**
This Work (SparseCommu)	**64K**	**Volta GPU x 512**	**56 Gbps**	**1.5 mins**	**58.2%**

ResNet

	Batch Size	Processor	GPU Interconnect	Time	Top-1 Accuracy
Goyal et al. [3]	8K	Pascal GPU x 256	56 Gbps	1 hour	76.3%
Smith et al. [41]	16K	Full TPU Pod	-	30 mins	76.1%
Codreanu et al. [42]	32K	KNL x 1024	-	42 mins	75.3%
You et al. [40]	32K	KNL x 2048	-	20 mins	75.4%
Akiba et al. [38]	32K	Pascal GPU x 1024	56 Gbps	15 mins	74.9%
Jia et al. [18]	64K	Pascal GPU x 1024	100 Gbps	8.7 mins	76.2%
Jia et al. [18]	64K	Pascal GPU x 2048	100 Gbps	6.6 mins	75.8%
Mikami et al. [43]	68K	Volta GPU x 2176	200 Gbps	3.7 mins	75.0%
This Work (DenseCommu)	64K	Volta GPU x 512	56 Gbps	7.3 mins	75.3%